Bridging Program Analysis and Large Language Models for Mobile App Security and Reliability

ACMRD 2026 - Germany

Muhammad Tehmasib Ali Tashfeen
Wichita State University
mxalitashfeen@shockers.wichita.edu

Security • Reliability • Program Analysis • LLMs

KEYNOTE MAP

From problem to research agenda

The story is organized around one question: what should each paradigm do best?

1. **Mobile App Analysis**
2. **Traditional Program Analysis**
3. **Expectation from LLMs**
4. **The bridge: constraints**
5. **Trends & challenges**

Key idea: use program analysis for evidence; use LLMs for interpretation; use constraints as the contract between them.

THE GAP

A field with two powerful tools — but no shared contract

Static analyzers know structure. LLMs know language and patterns. Mobile apps need both.

What program analysis sees

- control-flow and callbacks
- data-flow and taint
- lifecycle and typestate
- reachability and proof obligations

What LLMs see

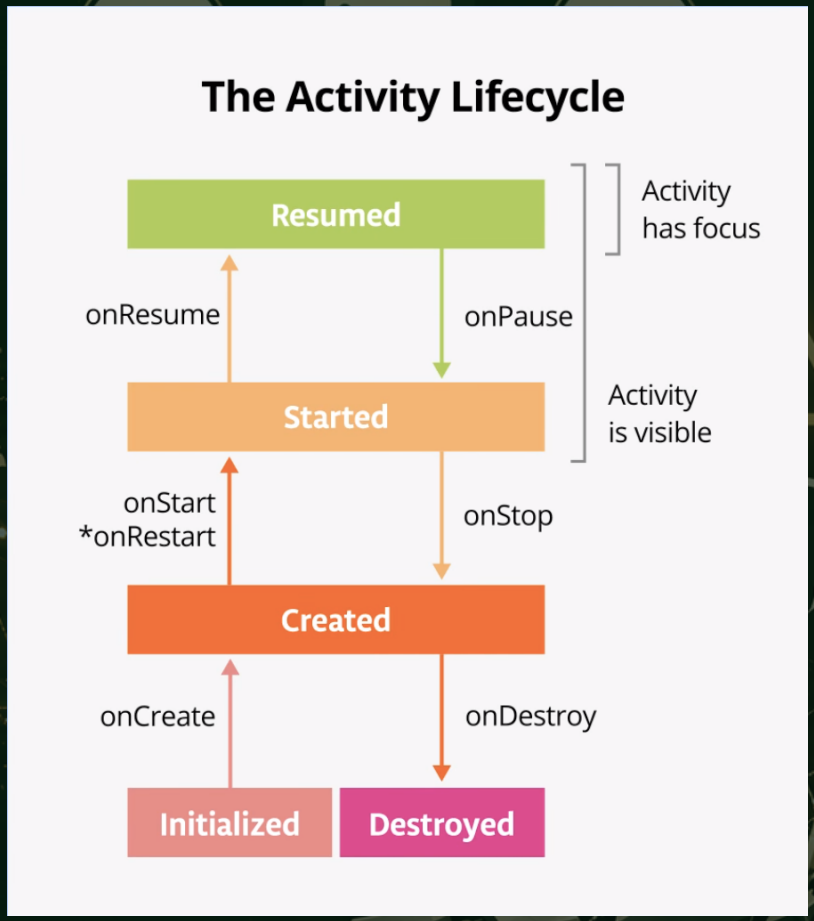
- API intent and documentation language
- idioms, exceptions, and incomplete code
- developer explanations and triage
- natural-language constraints

The missing center is not “more AI.” It is a rigorous interface between symbolic evidence and neural judgment.

MOBILE AS STRESS TEST

Why mobile apps break static assumptions

- Lifecycle
- Inter-component
- Resources
- Permissions



FAILURE SURFACE

Mobile assurance is not one bug class

Security and reliability failures often share the same root: violated platform constraints.



data exposure
insecure IPC
weak crypto
WebView risks

memory/resource leaks
lifecycle crashes
ANRs and battery drain
state inconsistency

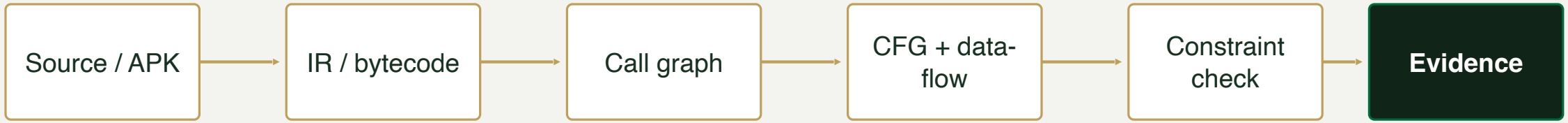
tampering
reverse engineering
debug artifacts
supply-chain drift

Design goal: detect violations before they become incidents — and explain them well enough to fix.

FOUNDATION I

Program analysis: turning code into evidence

A static analyzer does not “understand” code like a human. It computes facts over abstractions.



Strength

Repeatable, scalable, localizes facts, can prove absence under assumptions

Weakness

Approximation: callbacks, reflection, framework models, aliasing, libraries

Key discipline

Declare assumptions; return evidence; separate fact from confidence

FOUNDATION II

LLMs: powerful semantic assistants, not proof engines

LLMs work best when the task is constrained, evidence-grounded, and auditable.

Useful for

- mapping docs to rules
- explaining warnings
- prioritizing fixes
- detecting idioms and intent

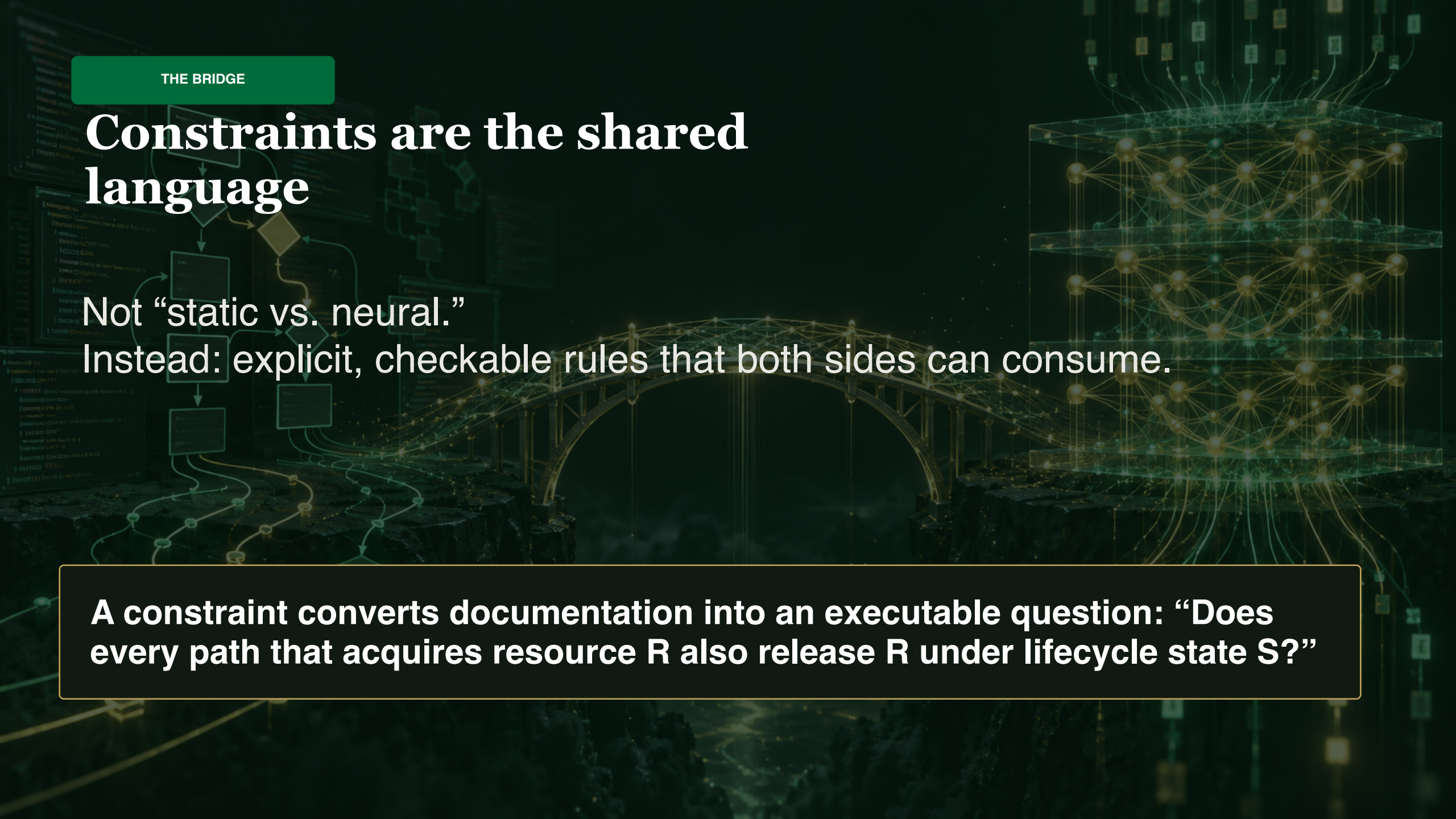
Risky for

- proving absence
- unseen app behavior
- precise alias reasoning
- security-critical authority

Best practice

- ground in static facts
- require quoted evidence
- use small schemas
- verify with tests/analyzers

LLM outputs should be treated as hypotheses — not final verdicts — until checked against code, constraints, and tests.



THE BRIDGE

Constraints are the shared language

Not “static vs. neural.”
Instead: explicit, checkable rules that both sides can consume.

A constraint converts documentation into an executable question: “Does every path that acquires resource R also release R under lifecycle state S?”

RUNNING EXAMPLE

Lifecycle constraint: services must terminate work

A small code pattern can create a large reliability problem.

Android / Java

```
1 class NotifyService extends Service {
2   @Override int onStartCommand(Intent i, int id) {
3     schedule(() -> {
4       createNotification();
5       // missing: stopSelf(id)
6     });
7     return START_STICKY;
8   }
9 }
```

Observation

The service starts asynchronous work and returns START_STICKY.

Evidence needed

Is there a feasible path from completion to stopSelf/stopService?

Possible impact

Lingering service → battery drain, memory pressure, poor reliability.

STATIC PASS

Predicate cascade: from app code to violation evidence

Static analysis should produce facts, not just a yes/no warning.



Executable rule schema

```
constraint ServiceMustTerminate(svc):
  for each onStartCommand entry e:
    if schedulesAsyncWork(e) or startsLongWork(e):
      paths = completionPaths(e)
      require exists p in paths: calls(p, stopSelf|stopService)
      emit evidence(entry, work, missingRelease, pathSlice)
```

LLM PASS

From raw warning to evidence-grounded judgment

LLMs should receive structured slices and return structured decisions.

Analyzer → LLM input

```
{
  "constraint": "ServiceMustTerminate",
  "entry": "NotifyService.onStartCommand",
  "evidence": [
    "finite async work completes",
    "no stopSelf/stopService call",
    "returns START_STICKY"
  ],
  "ask": "classify and explain"
}
```

LLM → analyzer output

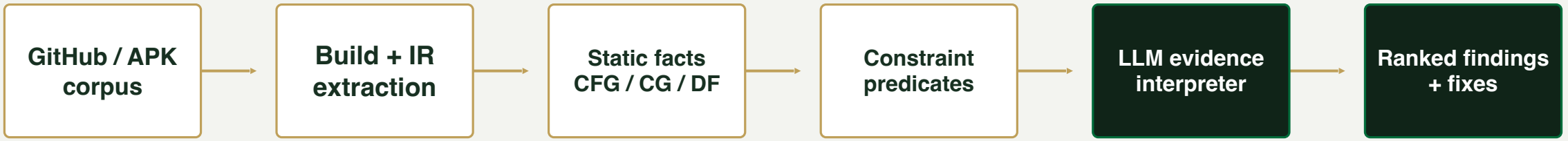
```
{
  "decision": "likely_violation",
  "confidence": 0.82,
  "rationale": "finite work lacks      termination",
  "repair_hint": "add stopSelf(id) or justify long-running service"
}
```

The LLM explains and prioritizes; the analyzer remains the source of concrete code evidence.

HYBRID ARCHITECTURE

A research pipeline for mobile assurance

Each component has a narrow responsibility and explicit artifacts.



Artifact	Repository metadata	IR, logs	Program facts	Rule result	Decision JSON	Report / patch
Failure mode	missing deps	unsupported bytecode	unsound model	overfit rule	hallucination	bad triage
Guardrail	rebuildable logs	version pinning	assumption log	unit tests	evidence schema	human review

TRUST MODEL

Do not let the LLM cross the authority boundary

A hybrid detector needs a governance model, not only an architecture diagram.

Analyzer authority

May assert: “this path contains call X; no release found under model M.”

Must expose assumptions and path evidence.

LLM authority

May assert: “this looks like finite work; this warning is likely actionable.”

Must cite input evidence, not hidden knowledge.

Human authority

May accept risk, reject false positive, rewrite code, or encode a new constraint.

Must receive understandable evidence.

Principle: neural reasoning can rank, explain, and hypothesize; symbolic analysis anchors the claim.

ANDROID SECURITY MAP

From standards to executable constraints

OWASP MASVS and MASWE provide a rich catalog; research can operationalize it.

Storage
encrypted data, backups, logs

Crypto
keys, RNG, deprecated algorithms

Auth
local auth, session termination

Network
TLS, trust, certificate validation

Platform
intents, WebViews, deep links

Code
dependencies, APIs, debug artifacts

Resilience
tamper/reverse engineering

Privacy
permissions, consent, data minimization

Research opportunity: automatically translate mobile security knowledge into analyzable, testable, and explainable obligations.

INDUSTRY TRENDS

Why this bridge matters now

Mobile teams are adopting AI assistance faster than their assurance workflows are adapting.

AI-assisted development

Code assistants, repo agents, code review bots, and multi-model orchestration are moving into everyday workflows.

Mobile standards maturity

MASVS/MASWE/MASTG are increasingly used as common vocabulary for mobile assurance.

Shift-left security

Secure development frameworks push security practices earlier in design, build, and test stages.

Platform fragmentation

Version drift, API behavior, libraries, and OEM/device differences continue to complicate evidence.

RESEARCH TRENDS

Where the field is moving

The next generation of tools will be hybrid, interactive, and benchmark-driven.

Neuro-symbolic analysis

LLMs guide search; static facts ground claims

Repository-scale agents

Plan, patch, test, and explain across projects

Spec mining from docs

Convert API rules into constraints and tests

Evidence-centric benchmarks

Evaluate findings, fixes, explanations, and false positives

The research frontier is not “LLM replaces analyzer.” It is “LLM and analyzer exchange evidence under constraints.”

CHALLENGES

The hard problems are not solved by larger models alone

A credible research agenda must make the limitations visible.

- **Soundness vs. precision**
- **Hallucination and authority**
- **Evaluation leakage**
- **Ecosystem drift**
- **Developer trust**
- **Security of AI pipeline**

RESEARCH AGENDA

Five questions for the next generation of mobile assurance

These questions can guide papers, tools, datasets, and doctoral research.

- **Q1** How do we mine platform documentation into executable constraints?
- **Q2** How can static analysis expose evidence that LLMs can safely interpret?
- **Q3** How do we benchmark explanation quality, not only detection accuracy?
- **Q4** How do we verify AI-generated fixes against lifecycle and security constraints?
- **Q5** How do we deploy hybrid detectors in CI without overwhelming developers?

Message: build tools that are scalable, explainable, and responsible — not merely impressive.

PROVOCATION

What should dependable engineers become?

We have frameworks for what dependable systems must do. We also need frameworks for what AI-assisted engineers must verify, question, and take responsibility for.

Govern AI

Do not merely use generated code; demand evidence, tests, and constraints.

Preserve judgment

Treat model output as a hypothesis; keep architectural and ethical reasoning human-led.

Build for community

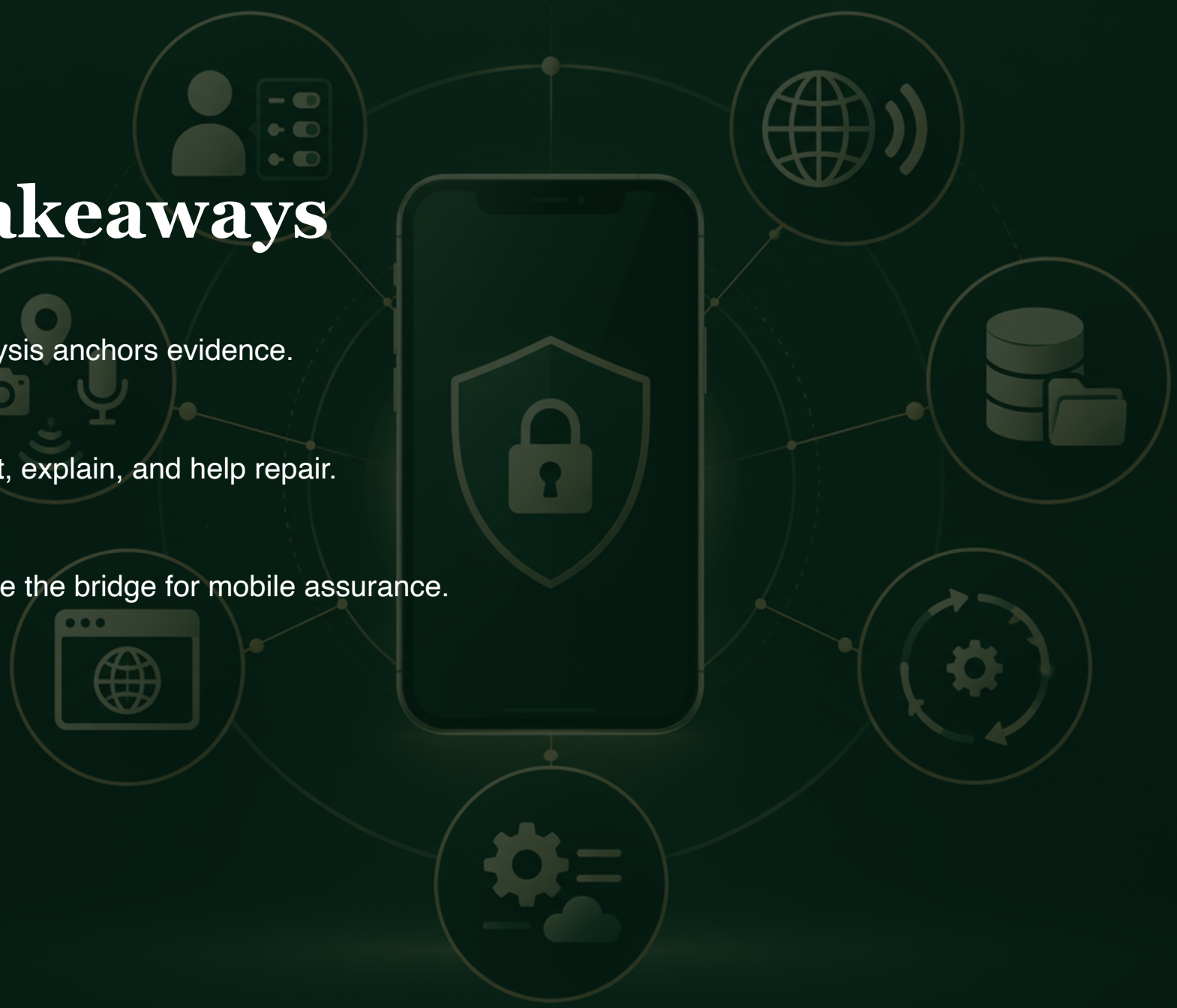
Mobile assurance protects users, data, devices, and trust in digital services.

The profession that bridges evidence and intelligence will not only survive AI — it will become more rigorous.

THANK YOU

Three takeaways

1. Program analysis anchors evidence.
2. LLMs interpret, explain, and help repair.
3. Constraints are the bridge for mobile assurance.



Questions?